MongoDB常见面试题总结（上）

Guide 数据库 NoSQL, MongoDB 约 5947 字 大约 20 分钟

> 少部分内容参考了 MongoDB 官方文档的描述，在此说明一下。

MongoDB 基础

MongoDB 是什么？

MongoDB 是一个基于 **分布式文件存储** 的开源 NoSQL 数据库系统，由 **C++** 编写的。MongoDB 提供了 **面向文档** 的存储方式，操作起来比较简单和容易，支持“**无模式**”的数据建模，可以存储比较复杂的数据类型，是一款非常流行的 **文档类型数据库** 。

在高负载的情况下，MongoDB 天然支持水平扩展和高可用，可以很方便地添加更多的节点/实例，以保证服务性能和可用性。在许多场景下，MongoDB 可以用于代替传统的关系型数据库或键/值存储方式，皆在为 Web 应用提供可扩展的高可用高性能数据存储解决方案。

MongoDB 的存储结构是什么？

MongoDB 的存储结构区别于传统的关系型数据库，主要由如下三个单元组成：

- **文档（Document）**：MongoDB 中最基本的单元，由 BSON 键值对（key-value）组成，类似于关系型数据库中的行（Row）。
- **集合（Collection）**：一个集合可以包含多个文档，类似于关系型数据库中的表（Table）。
- **数据库（Database）**：一个数据库中可以包含多个集合，可以在 MongoDB 中创建多个数据库，类似于关系型数据库中的数据库（Database）。

也就是说，MongoDB 将数据记录存储为文档 （更具体来说是BSON 文档 ），这些文档在集合中聚集在一起，数据库中存储一个或多个文档集合。

SQL 与 MongoDB 常见术语对比：

SQL	MongoDB
表（Table）	集合（Collection）

SQL	MongoDB
行（Row）	文档（Document）
列（Col）	字段（Field）
主键（Primary Key）	对象 ID（Objectid）
索引（Index）	索引（Index）
嵌套表（Embedded Table）	嵌入式文档（Embedded Document）
数组（Array）	数组（Array）

文档

MongoDB 中的记录就是一个 BSON 文档，它是由键值对组成的数据结构，类似于 JSON 对象，是 MongoDB 中的基本数据单元。字段的值可能包括其他文档、数组和文档数组。

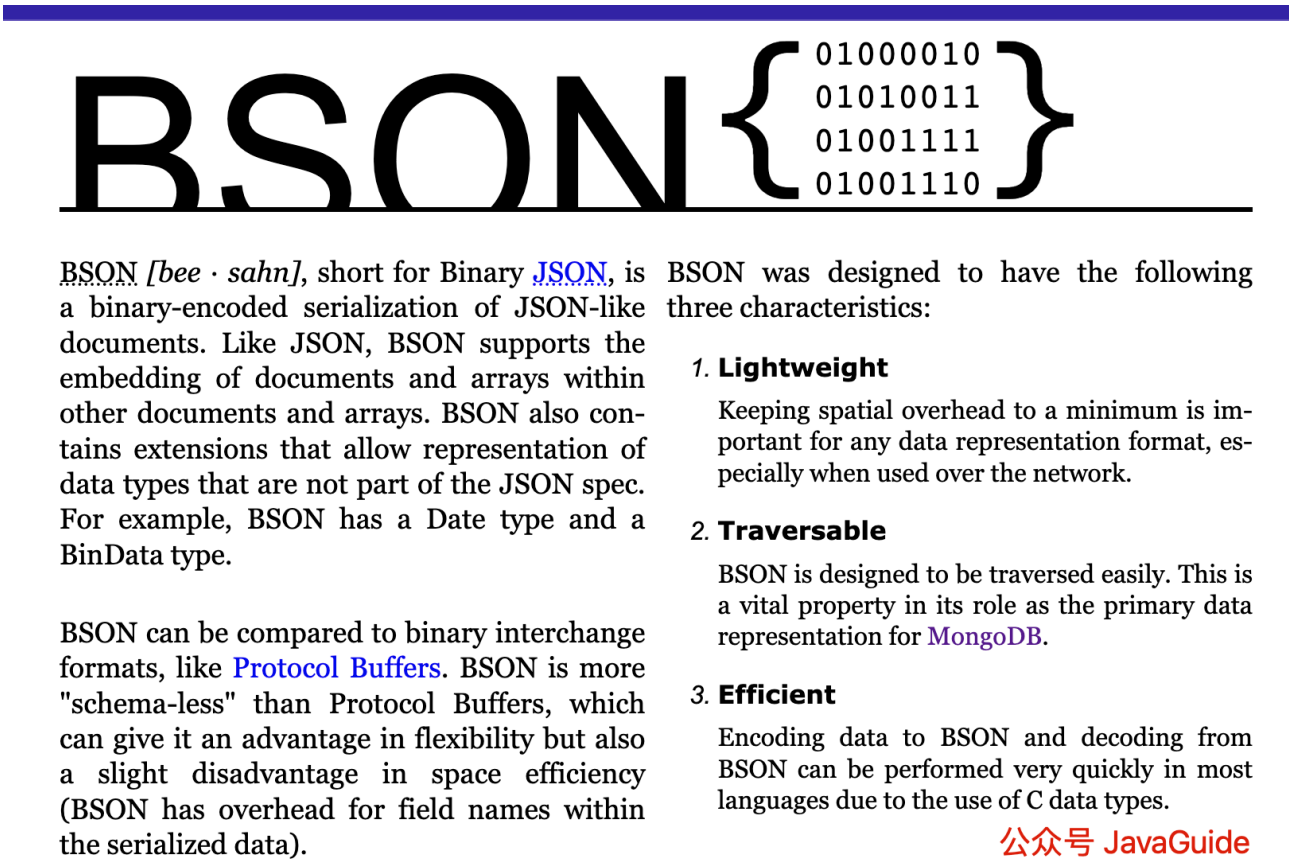
文档的键是字符串。除了少数例外情况，键可以使用任意 UTF-8 字符。

- 键不能含有 `\0` (空字符)。这个字符用来表示键的结尾。
- `.` 和 `$` 有特别的意义，只有在特定环境下才能使用。
- 以下划线 `_` 开头的键是保留的(不是严格要求的)。

BSON [bee·sahn] 是 Binary JSON 的简称，是 JSON 文档的二进制表示，支持将文档和数组嵌入到其他文档和数组中，还包含允许表示不属于 JSON 规范的数据类型的扩展。有关 BSON 规范的内容，可以参考 bsonspec.org ，另见BSON 类型 。

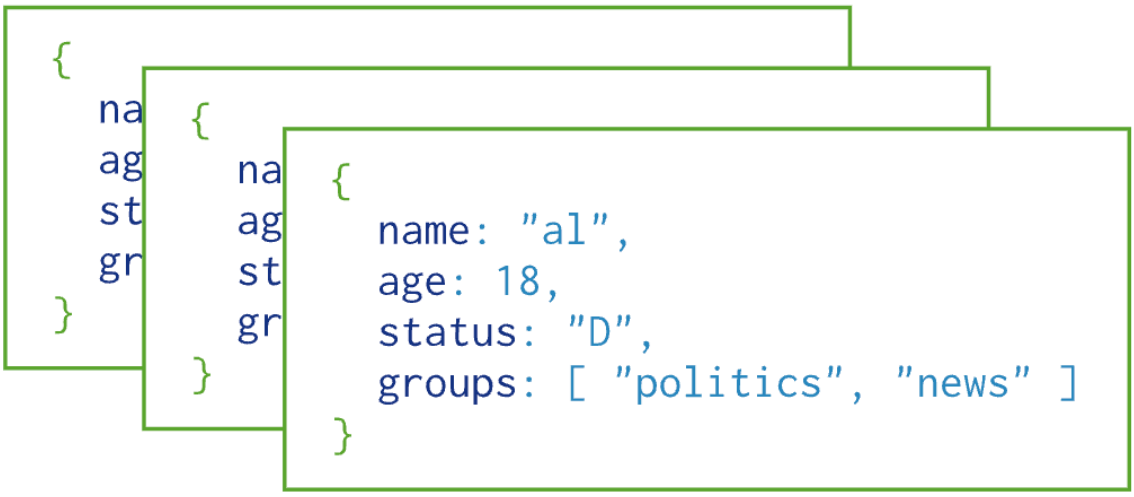
根据维基百科对 BJSON 的介绍，BJSON 的遍历速度优于 JSON，这也是 MongoDB 选择 BSON 的主要原因，但 BJSON 需要更多的存储空间。

> 与 JSON 相比，BSON 着眼于提高存储和扫描效率。BSON 文档中的大型元素以长度字段为前缀以便于扫描。在某些情况下，由于长度前缀和显式数组索引的存在，BSON 使用的空间会多于 JSON。



集合

MongoDB 集合存在于数据库中，**没有固定的结构**，也就是 **无模式** 的，这意味着可以往集合插入不同格式和类型的数据。不过，通常情况下，插入集合中的数据都会有一定的关联性。



集合不需要事先创建，当第一个文档插入或者第一个索引创建时，如果该集合不存在，则会创建一个新的集合。

集合名可以是满足下列条件的任意 UTF-8 字符串：

- 集合名不能是空字符串 `""` 。
- 集合名不能含有 `\0` （空字符)，这个字符表示集合名的结尾。
- 集合名不能以"system."开头，这是为系统集合保留的前缀。例如 `system.users` 这个集合保存着数据库的用户信息， `system.namespaces` 集合保存着所有数据库集合的信息。
- 集合名必须以下划线或者字母符号开始，并且不能包含 `$` 。

数据库

数据库用于存储所有集合，而集合又用于存储所有文档。一个 MongoDB 中可以创建多个数据库，每一个数据库都有自己的集合和权限。

MongoDB 预留了几个特殊的数据库。

- **admin** : admin 数据库主要是保存 root 用户和角色。例如，system.users 表存储用户，system.roles 表存储角色。一般不建议用户直接操作这个数据库。将一个用户添加到这个数据库，且使它拥有 admin 库上的名为 dbAdminAnyDatabase 的角色权限，这个用户自动继承所有数据库的权限。一些特定的服务器端命令也只能从这个数据库运行，比如关闭服务器。
- **local** : local 数据库是不会被复制到其他分片的，因此可以用来存储本地单台服务器的任意 collection。一般不建议用户直接使用 local 库存储任何数据，也不建议进行 CRUD 操作，因为数据无法被正常备份与恢复。
- **config** : 当 MongoDB 使用分片设置时，config 数据库可用来保存分片的相关信息。
- **test** : 默认创建的测试库，连接 mongod 服务时，如果不指定连接的具体数据库，默认就会连接到 test 数据库。

数据库名可以是满足以下条件的任意 UTF-8 字符串：

- 不能是空字符串 `""` 。
- 不得含有 `' '` （空格)、 `.` 、 `$` 、 `/` 、 `\` 和 `\0` (空字符)。
- 应全部小写。
- 最多 64 字节。

数据库名最终会变成文件系统里的文件，这也就是有如此多限制的原因。

MongoDB 有什么特点?

- **数据记录被存储为文档**：MongoDB 中的记录就是一个 BSON 文档，它是由键值对组成的数据结构，类似于 JSON 对象，是 MongoDB 中的基本数据单元。
- **模式自由**：集合的概念类似 MySQL 里的表，但它不需要定义任何模式，能够用更少的数据对象表现复杂的领域模型对象。
- **支持多种查询方式**：MongoDB 查询 API 支持读写操作 (CRUD)以及数据聚合、文本搜索和地理空间查询。
- **支持 ACID 事务**：NoSQL 数据库通常不支持事务，为了可扩展和高性能进行了权衡。不过，也有例外，MongoDB 就支持事务。与关系型数据库一样，MongoDB 事务同样具有 ACID 特性。MongoDB 单文档原生支持原子性，也具备事务的特性。MongoDB 4.0 加入了对多文档事务的支持，但只支持复制集部署模式下的事务，也就是说事务的作用域限制为一个副本集内。MongoDB 4.2 引入了分布式事务，增加了对分片集群上多文档事务的支持，并合并了对副本集上多文档事务的现有支持。
- **高效的二进制存储**：存储在集合中的文档，是以键值对的形式存在的。键用于唯一标识一个文档，一般是 ObjectId 类型，值是以 BSON 形式存在的。BSON = Binary JSON，是在 JSON 基础上加了一些类型及元数据描述的格式。
- **自带数据压缩功能**：存储同样的数据所需的资源更少。
- **支持 mapreduce**：通过分治的方式完成复杂的聚合任务。不过，从 MongoDB 5.0 开始，map-reduce 已经不被官方推荐使用了，替代方案是 聚合管道 。聚合管道提供比 map-reduce 更好的性能和可用性。
- **支持多种类型的索引**：MongoDB 支持多种类型的索引，包括单字段索引、复合索引、多键索引、哈希索引、文本索引、 地理位置索引等，每种类型的索引有不同的使用场合。
- **支持 failover**：提供自动故障恢复的功能，主节点发生故障时，自动从从节点中选举出一个新的主节点，确保集群的正常使用，这对于客户端来说是无感知的。
- **支持分片集群**：MongoDB 支持集群自动切分数据，让集群存储更多的数据，具备更强的性能。在数据插入和更新时，能够自动路由和存储。
- **支持存储大文件**：MongoDB 的单文档存储空间要求不超过 16MB。对于超过 16MB 的大文件，MongoDB 提供了 GridFS 来进行存储，通过 GridFS，可以将大型数据进行分块处理，然后将这些切分后的小文档保存在数据库中。

MongoDB 适合什么应用场景？

MongoDB 的优势在于其数据模型和存储引擎的灵活性、架构的可扩展性以及对强大的索引支持。

选用 MongoDB 应该充分考虑 MongoDB 的优势，结合实际项目的需求来决定：

- 随着项目的发展，使用类 JSON 格式（BSON）保存数据是否满足项目需求？MongoDB 中的记录就是一个 BSON 文档，它是由键值对组成的数据结构，类似于 JSON 对象，是 MongoDB 中的基本数据单元。
- 是否需要大数据量的存储？是否需要快速水平扩展？MongoDB 支持分片集群，可以很方便地添加更多的节点（实例），让集群存储更多的数据，具备更强的性能。
- 是否需要更多类型索引来满足更多应用场景？MongoDB 支持多种类型的索引，包括单字段索引、复合索引、多键索引、哈希索引、文本索引、 地理位置索引等，每种类型的索引有不同的使用场合。
- ……

MongoDB 存储引擎

MongoDB 支持哪些存储引擎？

存储引擎（Storage Engine）是数据库的核心组件，负责管理数据在内存和磁盘中的存储方式。

与 MySQL 一样，MongoDB 采用的也是 **插件式的存储引擎架构** ，支持不同类型的存储引擎，不同的存储引擎解决不同场景的问题。在创建数据库或集合时，可以指定存储引擎。

> 插件式的存储引擎架构可以实现 Server 层和存储引擎层的解耦，可以支持多种存储引擎，如 MySQL 既可以支持 B-Tree 结构的 InnoDB 存储引擎，还可以支持 LSM 结构的 RocksDB 存储引擎。

在存储引擎刚出来的时候，默认是使用 MMAPV1 存储引擎，MongoDB4.x 版本不再支持 MMAPv1 存储引擎。

现在主要有下面这两种存储引擎：

- **WiredTiger 存储引擎**：自 MongoDB 3.2 以后，默认的存储引擎为 WiredTiger 存储引擎 。非常适合大多数工作负载，建议用于新部署。WiredTiger 提供文档级并发模型、

检查点和数据压缩（后文会介绍到）等功能。
- **In-Memory 存储引擎**：In-Memory 存储引擎 在 MongoDB Enterprise 中可用。它不是将文档存储在磁盘上，而是将它们保留在内存中以获得更可预测的数据延迟。

此外，MongoDB 3.0 提供了 **可插拔的存储引擎 API** ，允许第三方为 MongoDB 开发存储引擎，这点和 MySQL 也比较类似。

WiredTiger 基于 LSM Tree 还是 B+ Tree?

目前绝大部分流行的数据库存储引擎都是基于 B/B+ Tree 或者 LSM(Log Structured Merge) Tree 来实现的。对于 NoSQL 数据库来说，绝大部分（比如 HBase、Cassandra、RocksDB）都是基于 LSM 树，MongoDB 不太一样。

上面也说了，自 MongoDB 3.2 以后，默认的存储引擎为 WiredTiger 存储引擎。在 WiredTiger 引擎官网上，我们发现 WiredTiger 使用的是 B+ 树作为其存储结构：

```plain
WiredTiger maintains a table's data in memory using a data structure called a B-Tree ( B+ Tree to be specific), referring to the nodes of a B-Tree as pages. Internal pages carry only keys. The leaf pages store both keys and values.
```

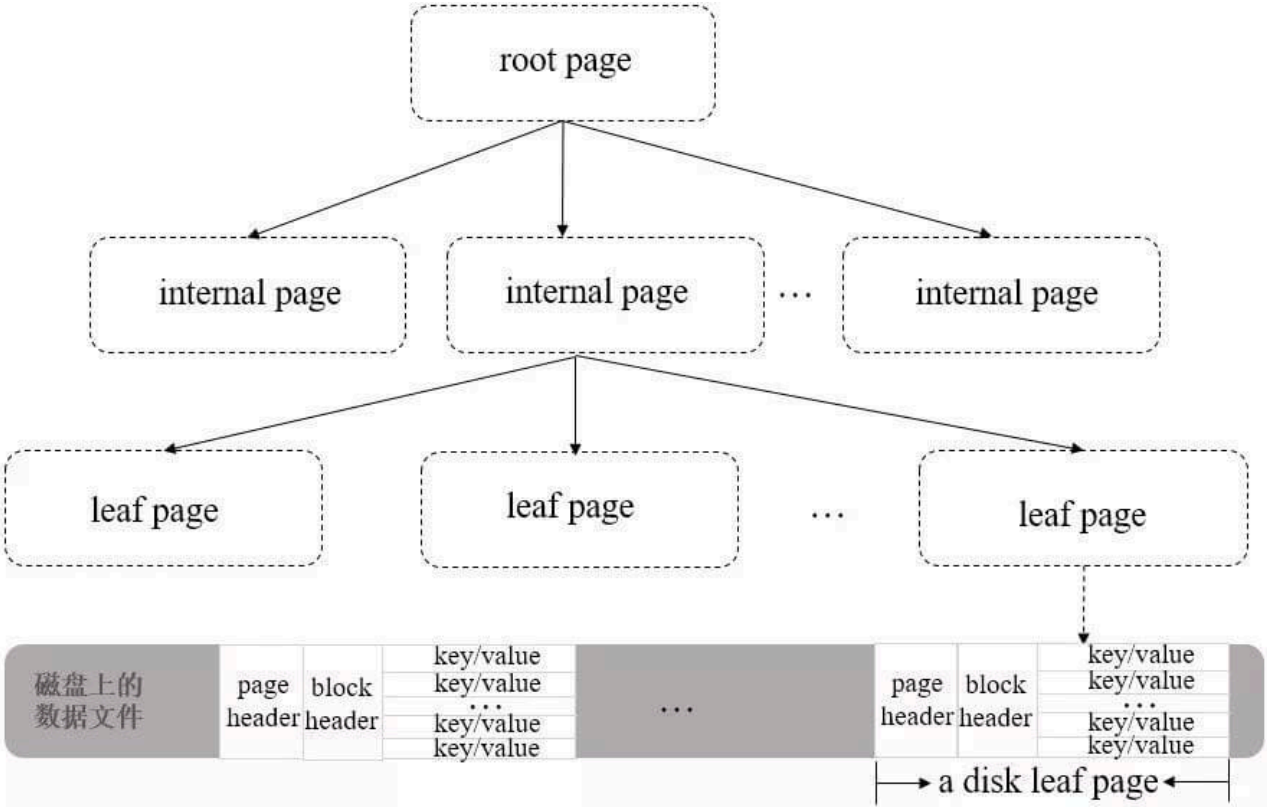
此外，WiredTiger 还支持 LSM(Log Structured Merge) 树作为存储结构，MongoDB 在使用 WiredTiger 作为存储引擎时，默认使用的是 B+ 树。

如果想要了解 MongoDB 使用 B+ 树的原因，可以看看这篇文章：【驳斥八股文系列】别瞎分析了，MongoDB 使用的是 B+ 树，不是你们以为的 B 树 。

使用 B+ 树时，WiredTiger 以 **page** 为基本单位往磁盘读写数据。B+ 树的每个节点为一个 page，共有三种类型的 page：

- **root page（根节点）**：B+ 树的根节点。
- **internal page（内部节点）**：不实际存储数据的中间索引节点。
- **leaf page（叶子节点）**：真正存储数据的叶子节点，包含一个页头（page header）、块头（block header）和真正的数据（key/value），其中页头定义了页的类型、页中实际载荷数据的大小、页中记录条数等信息；块头定义了此页的 checksum、块在磁盘上的寻址位置等信息。

其整体结构如下图所示：



如果想要深入研究学习 WiredTiger 存储引擎，推荐阅读 MongoDB 中文社区的 WiredTiger 存储引擎系列 。

MongoDB 聚合

MongoDB 聚合有什么用?

实际项目中，我们经常需要将多个文档甚至是多个集合汇总到一起计算分析（比如求和、取最大值）并返回计算后的结果，这个过程被称为 **聚合操作** 。

根据官方文档介绍，我们可以使用聚合操作来：

- 将来自多个文档的值组合在一起。
- 对集合中的数据进行的一系列运算。
- 分析数据随时间的变化。

MongoDB 提供了哪几种执行聚合的方法?

MongoDB 提供了两种执行聚合的方法：

- **聚合管道（Aggregation Pipeline）**：执行聚合操作的首选方法。

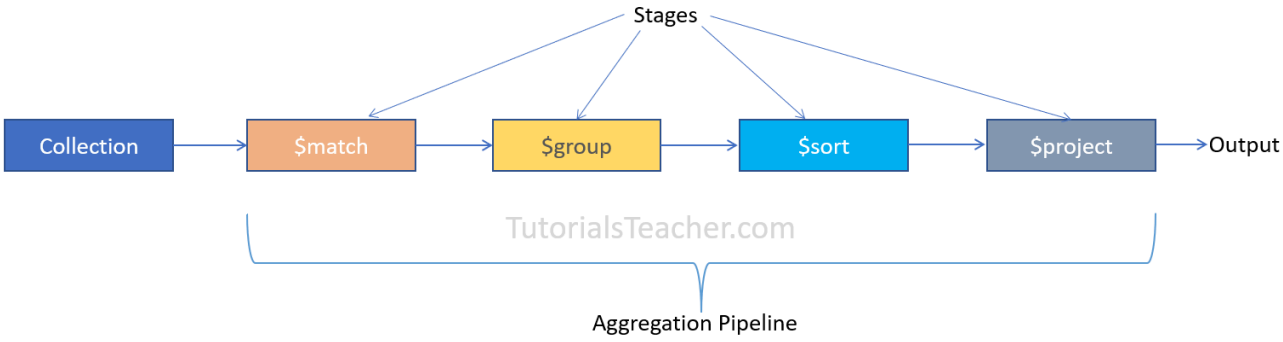
- **单一目的聚合方法（Single purpose aggregation methods）**：也就是单一作用的聚合函数比如 `count()` 、 `distinct()` 、 `estimatedDocumentCount()` 。

绝大部分文章中还提到了 **map-reduce** 这种聚合方法。不过，从 MongoDB 5.0 开始，map-reduce 已经不被官方推荐使用了，替代方案是 聚合管道 。聚合管道提供比 map-reduce 更好的性能和可用性。

MongoDB 聚合管道由多个阶段组成，每个阶段在文档通过管道时转换文档。每个阶段接收前一个阶段的输出，进一步处理数据，并将其作为输入数据发送到下一个阶段。

每个管道的工作流程是：

1. 接受一系列原始数据文档
2. 对这些文档进行一系列运算
3. 结果文档输出给下一个阶段



常用阶段操作符：

操作符	简述
$match	匹配操作符，用于对文档集合进行筛选
$project	投射操作符，用于重构每一个文档的字段，可以提取字段，重命名字段，甚至可以对原有字段进行操作后新增字段
$sort	排序操作符，用于根据一个或多个字段对文档进行排序
$limit	限制操作符，用于限制返回文档的数量
$skip	跳过操作符，用于跳过指定数量的文档
$count	统计操作符，用于统计文档的数量
$group	分组操作符，用于对文档集合进行分组

操作符	简述
$unwind	拆分操作符，用于将数组中的每一个值拆分为单独的文档
$lookup	连接操作符，用于连接同一个数据库中另一个集合，并获取指定的文档，类似于 populate

更多操作符介绍详见官方文档：
https://docs.mongodb.com/manual/reference/operator/aggregation/

阶段操作符用于 `db.collection.aggregate` 方法里面，数组参数中的第一层。

```sql
db.collection.aggregate( [ { 阶段操作符：表述 }, { 阶段操作符：表述 }, ... ] )
```

下面是 MongoDB 官方文档中的一个例子：

```sql
db.orders.aggregate([
   # 第一阶段：$match阶段按status字段过滤文档，并将status等于"A"的文档传递到下一阶段。
    { $match: { status: "A" } },
  # 第二阶段：$group阶段按cust_id字段将文档分组，以计算每个cust_id唯一值的金额总和。
    { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }
])
```

MongoDB 事务

> MongoDB 事务想要搞懂原理还是比较花费时间的，我自己也没有搞太明白。因此，我这里只是简单介绍一下 MongoDB 事务，想要了解原理的小伙伴，可以自行搜索查阅相关资料。
> 这里推荐几篇文章，供大家参考：
>
> - 技术干货| MongoDB 事务原理
> - MongoDB 一致性模型设计与实现
> - MongoDB 官方文档对事务的介绍

我们在介绍 NoSQL 数据的时候也说过，NoSQL 数据库通常不支持事务，为了可扩展和高性能进行了权衡。不过，也有例外，MongoDB 就支持事务。

与关系型数据库一样，MongoDB 事务同样具有 ACID 特性：

- **原子性**（`Atomicity`）：事务是最小的执行单位，不允许分割。事务的原子性确保动作要么全部完成，要么完全不起作用；
- **一致性**（`Consistency`）：执行事务前后，数据保持一致，例如转账业务中，无论事务是否成功，转账者和收款人的总额应该是不变的；
- **隔离性**（`Isolation`）：并发访问数据库时，一个用户的事务不被其他事务所干扰，各并发事务之间数据库是独立的。WiredTiger 存储引擎支持读未提交（read-uncommitted）、读已提交（read-committed）和快照（snapshot）隔离，MongoDB 启动时默认选快照隔离。在不同隔离级别下，一个事务的生命周期内，可能出现脏读、不可重复读、幻读等现象。
- **持久性**（`Durability`）：一个事务被提交之后。它对数据库中数据的改变是持久的，即使数据库发生故障也不应该对其有任何影响。

关于事务的详细介绍这篇文章就不多说了，感兴趣的可以看看我写的MySQL 常见面试题总结这篇文章，里面有详细介绍到。

MongoDB 单文档原生支持原子性，也具备事务的特性。当谈论 MongoDB 事务的时候，通常指的是 **多文档** 。MongoDB 4.0 加入了对多文档 ACID 事务的支持，但只支持复制集部署模式下的 ACID 事务，也就是说事务的作用域限制为一个副本集内。MongoDB 4.2 引入了 **分布式事务** ，增加了对分片集群上多文档事务的支持，并合并了对副本集上多文档事务的现有支持。

根据官方文档介绍：

> 从 MongoDB 4.2 开始，分布式事务和多文档事务在 MongoDB 中是一个意思。分布式事务是指分片集群和副本集上的多文档事务。从 MongoDB 4.2 开始，多文档事务（无论是在分片集群还是副本集上）也称为分布式事务。

在大多数情况下，多文档事务比单文档写入会产生更大的性能成本。对于大部分场景来说，非规范化数据模型（嵌入式文档和数组） 依然是最佳选择。也就是说，适当地对数据进行建模可以最大限度地减少对多文档事务的需求。

注意：

- 从 MongoDB 4.2 开始，多文档事务支持副本集和分片集群，其中：主节点使用 WiredTiger 存储引擎，同时从节点使用 WiredTiger 存储引擎或 In-Memory 存储引擎。在 MongoDB 4.0 中，只有使用 WiredTiger 存储引擎的副本集支持事务。
- 在 MongoDB 4.2 及更早版本中，你无法在事务中创建集合。从 MongoDB 4.4 开始，您可以在事务中创建集合和索引。有关详细信息，请参阅 在事务中创建集合和索引 。

MongoDB 数据压缩

借助 WiredTiger 存储引擎（MongoDB 3.2 后的默认存储引擎），MongoDB 支持对所有集合和索引进行压缩。压缩以额外的 CPU 为代价最大限度地减少存储使用。

默认情况下，WiredTiger 使用 Snappy 压缩算法（谷歌开源，旨在实现非常高的速度和合理的压缩，压缩比 3 ～ 5 倍）对所有集合使用块压缩，对所有索引使用前缀压缩。

除了 Snappy 之外，对于集合还有下面这些压缩算法：

- zlib ：高度压缩算法，压缩比 5 ～ 7 倍
- Zstandard （简称 zstd）：Facebook 开源的一种快速无损压缩算法，针对 zlib 级别的实时压缩场景和更好的压缩比，提供更高的压缩率和更低的 CPU 使用率，MongoDB 4.2 开始可用。

WiredTiger 日志也会被压缩，默认使用的也是 Snappy 压缩算法。如果日志记录小于或等于 128 字节，WiredTiger 不会压缩该记录。

Amazon Document 与 MongoDB 的差异

Amazon DocumentDB（与 MongoDB 兼容） 是一种快速、可靠、完全托管的数据库服务。Amazon DocumentDB 可在云中轻松设置、操作和扩展与 MongoDB 兼容的数据库。

`$vectorSearch` 运算符

Amazon DocumentDB 不支持 `$vectorSearch` 作为独立运营商。相反，我们在 `$search` 运营商 `vectorSearch` 内部支持。有关更多信息，请参阅 向量搜索 Amazon DocumentDB 。

`OpCountersCommand`

Amazon DocumentDB 的 `OpCountersCommand` 行为偏离于 MongoDB 的 `opcounters.command` 如下：

- MongoDB 的 `opcounters.command` 计入除插入、更新和删除之外的所有命令，而 Amazon DocumentDB 的 `OpCountersCommand` 也排除 `find` 命令。

- Amazon DocumentDB 将内部命令（例如 `getCloudWatchMetricsV2` ）对 `OpCountersCommand` 计入。

管理数据库和集合

Amazon DocumentDB 不支持管理或本地数据库，MongoDB `system.*` 或 `startup_log` 集合也不支持。

`cursormaxTimeMS`

在 Amazon DocumentDB 中， `cursor.maxTimeMS` 重置每个请求的计数器。 `getMore` 因此，如果指定了 3000MS `maxTimeMS` ，则该查询耗时 2800MS，而每个后续 `getMore` 请求耗时 300MS，则游标不会超时。游标仅在单个操作（无论是查询还是单个 `getMore` 请求）耗时超过指定值时才将超时 `maxTimeMS` 。此外，检查游标执行时间的扫描器以五 (5) 分钟间隔尺寸运行。

explain()

Amazon DocumentDB 在利用分布式、容错、自修复的存储系统的专用数据库引擎上模拟 MongoDB 4.0 API。因此，查询计划和 `explain()` 的输出在 Amazon DocumentDB 和 MongoDB 之间可能有所不同。希望控制其查询计划的客户可以使用 `$hint` 运算符强制选择首选索引。

字段名称限制

Amazon DocumentDB 不支持点“。” 例如，文档字段名称中 `db.foo.insert({‘x.1’:1})` 。

Amazon DocumentDB 也不支持字段名称中的 $ 前缀。

例如，在 Amazon DocumentDB 或 MongoDB 中尝试以下命令：

```shell
rs0:PRIMARY< db.foo.insert({"a":{"$a":1}})
```

MongoDB 将返回以下内容：

```shell
WriteResult({ "nInserted" : 1 })
```

Amazon DocumentDB 将返回一个错误：

```shell
WriteResult({
  "nInserted" : 0,
  "writeError" : {
    "code" : 2,
    "errmsg" : "Document can't have $ prefix field names: $a"
  }
})
```

参考

- MongoDB 官方文档（主要参考资料，以官方文档为准）：https://www.mongodb.com/docs/manual/
- 《MongoDB 权威指南》
- 技术干货| MongoDB 事务原理 - MongoDB 中文社区：https://mongoing.com/archives/82187
- Transactions - MongoDB 官方文档：https://www.mongodb.com/docs/manual/core/transactions/
- WiredTiger Storage Engine - MongoDB 官方文档：https://www.mongodb.com/docs/manual/core/wiredtiger/
- WiredTiger 存储引擎之一：基础数据结构分析：https://mongoing.com/topic/archives-35143

JavaGuide官方公众号
（微信搜索JavaGuide）

1、公众号后台回复“PDF”获取原创PDF面试手册

2、公众号后台回复“学习路线”获取Java学习路线最新版

3、公众号后台回复“开源”获取优质Java开源项目合集

4、公众号后台回复“八股文”获取Java面试真题+面经

最近更新2024/4/10 00:20

贡献者: guide,Guide,Erzbir,Mr.Hope,wei.zou

Copyright © 2025 Guide